AA 1 — Análisis de actores y fundamentos de SDD

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Jaime Landázuri, Ivett Zaragocín, Alejandro Padilla, Doménica Rojas, Melissa Contento
Asignatura	Desarrollo Basado en Plataformas
Ciclo	5 A
Unidad	Unidad 1
Práctica Nro.	1
Tipo	Grupal
Título de la Práctica	Análisis de actores y fundamentos de Spec-Driven Development: Sistema de gestión de prácticas preprofesionales
Nombre del Docente	Ing. Edison Coronel
Fecha	8 de octubre de 2026

2. Identificación del proyecto

Analizamos el proyecto 7 del banco académico, **Sistema de gestión de prácticas preprofesionales** (complejidad media). Su propósito es apoyar la planificación, asignación, seguimiento y cierre de las prácticas, y relacionar a estudiantes, entidades receptoras, tutores, períodos, actividades y evidencias. El reto que plantea el banco es que el sistema no se limite a almacenar formularios: su valor está en el seguimiento y la trazabilidad.

En la ACD1 el equipo acordó un alcance MVP: desde la práctica ya asignada hasta su cierre. Quedan fuera la postulación, los convenios con las entidades y la integración con el SIAAF.

3. Mapa de actores

La figura muestra el sistema con los cinco momentos del proceso: arriba, los usuarios que lo operan; abajo, los actores externos y el tutor de la entidad; al pie, los servicios relacionados. Cada caja dice qué hace el actor y cada flecha, el sentido de la interacción. También aparecen dos relaciones entre actores: el coordinador designa al tutor académico y la entidad designa a su propio tutor.

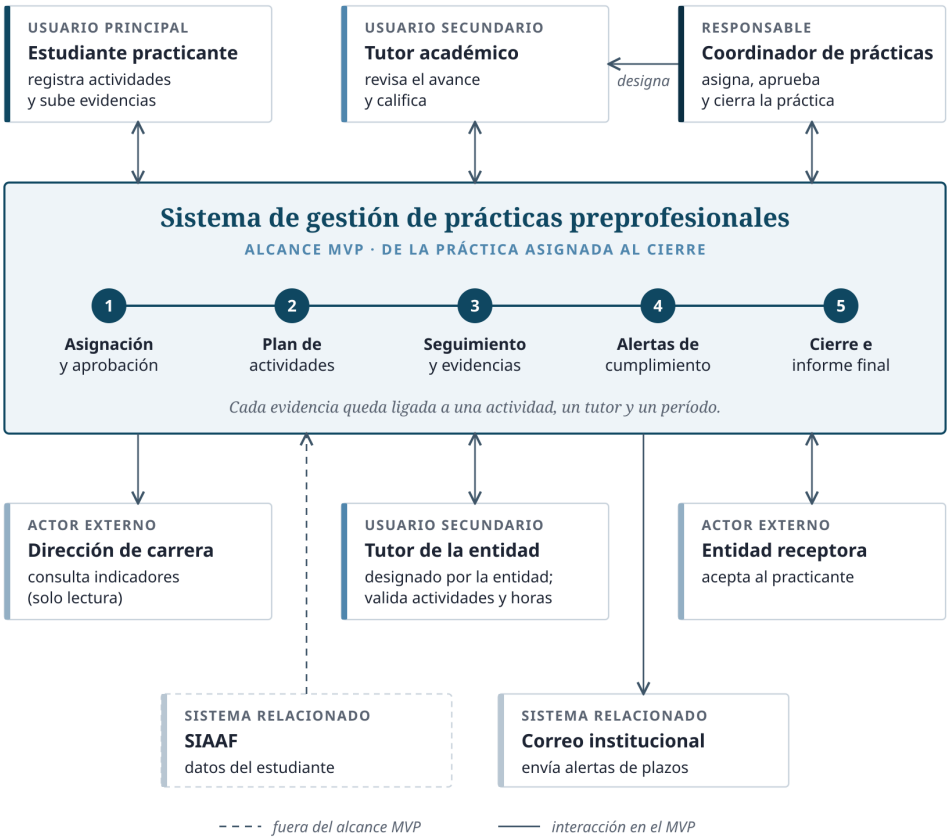


Figura 1: Mapa de actores del sistema de gestión de prácticas preprofesionales. Fuente: elaboración propia.

4. Caracterización de actores

Para identificar a los actores seguimos el criterio de Sharp et al. [1]: partir de los usuarios directos y ampliar hacia quienes los supervisan, los proveen de información o consumen sus resultados.

Actor	Clasificación	Necesidad o interés	Interacción con la solución	Información que proporciona o consulta	Restricciones o condiciones
Estudiante practicante	Usuario principal	Cumplir las horas y aprobar la práctica sin trámites en papel	Registra actividades, sube evidencias y consulta observaciones	Proporciona bitácora, horas y evidencias; consulta estado y calificación	Solo ve su propia práctica; plazos por período
Tutor académico	Usuario secundario	Seguir el avance de varios practicantes a la vez	Revisa evidencias, deja observaciones y califica	Consulta avance; proporciona observaciones y nota	Asignado por el coordinador; tiempo limitado
Tutor de la entidad	Usuario secundario	Validar lo que el estudiante hizo realmente	Aprueba actividades y horas	Proporciona validaciones y evaluación de desempeño	Externo a la UNL: acceso restringido a sus practicantes
Coordinador de prácticas	Responsable o administrador	Controlar el proceso de todos los practicantes de la carrera	Asigna práctica y tutores, aprueba el plan y cierra la práctica	Proporciona asignaciones y períodos; consulta reportes y alertas	Debe respetar el reglamento de prácticas
Entidad receptora	Actor externo	Recibir practicantes útiles a su operación	Acepta al practicante y designa a su tutor	Proporciona cupos, datos de la entidad y del tutor	El convenio se gestiona fuera del sistema (fuera del MVP)
Dirección de carrera	Actor externo (indirecto)	Contar con evidencia para seguimiento y acreditación	Consulta indicadores, sin operar el proceso	Consulta prácticas cerradas, horas y resultados	Solo lectura
SIAAF	Sistema o servicio relacionado	Ser la fuente oficial de datos académicos	En el MVP sus datos se cargan manualmente	Proporciona datos del estudiante y su matrícula	Sin integración en el MVP
Correo institucional	Sistema o servicio relacionado	Hacer llegar los avisos a tiempo	Recibe del sistema las alertas de plazos	Transporta notificaciones	Depende de cuentas institucionales activas

5. Fundamentos de Spec-Driven Development

Desarrollar guiado por especificaciones significa que la especificación, y no el código, es el artefacto principal: describe qué debe hacer el sistema y por qué, y el código se genera o se verifica contra ella [2].

Concepto	Qué es	Ejemplo en este proyecto
Problema	La situación del mundo real que se quiere cambiar	Las prácticas se controlan con formularios sueltos y no hay trazabilidad
Requisito	Una condición que la solución debe cumplir para resolver el problema	El tutor de la entidad debe validar cada actividad antes de que cuente como horas
Especificación	El conjunto de requisitos escrito de forma explícita y verificable	spec.md con historias de usuario y criterios de aceptación
Solución técnica	Cómo se construye: stack, arquitectura, datos	API REST, base de datos relacional, almacenamiento de archivos

Zave y Jackson muestran que los requisitos se expresan en el dominio del problema, no en el de la máquina [3]. Por eso la especificación se escribe antes de implementar: fija el qué, sirve de contrato y define cómo se comprobará el resultado.

La clarificación importa porque una ambigüedad no resuelta antes de planificar la termina decidiendo quien programa, o un asistente de IA, sin validación [4].

Especificación, planificación y tareas forman una cadena: el plan traduce la especificación a decisiones técnicas y las tareas parten el plan en unidades verificables, cada una rastreable a un requisito.

6. Relación con GitHub Spec Kit

Spec Kit implementa este flujo como comandos que el asistente ejecuta en orden [5]:

Comando	Función	Artefacto	Relación con SDD
constitution	Fija los principios del proyecto con los que se evalúa cada fase	constitution.md	Reglas que toda especificación y todo plan deben respetar
specify	Describe el qué y el porqué, sin tecnología	spec.md	Es la especificación: problema, actores, historias y criterios
clarify	Hace hasta cinco preguntas sobre lo ambiguo y escribe las respuestas en la spec	spec.md actualizado	La clarificación antes de diseñar
plan	Decide stack, arquitectura y modelo de datos a partir de la spec	plan.md, data-model.md	La solución técnica, ya separada del problema
tasks	Parte el plan en tareas ordenadas y accionables	tasks.md	Unidades de trabajo rastreables a requisitos

El orden importa: specify es el único paso obligatorio antes de plan, pero saltear clarify implica diseñar sobre ambigüedades.

7. Preguntas de clarificación

1. ¿Quién aprueba el plan de actividades: el coordinador, el tutor académico o ambos, y en qué orden?
2. ¿Qué evidencia mínima debe acompañar cada actividad registrada (foto, documento firmado, informe), y quién la valida primero?
3. ¿Qué pasa si el tutor de la entidad no valida actividades dentro del plazo: se escala al coordinador o vence automáticamente?
4. ¿Qué alertas necesita cada actor y con qué anticipación (horas pendientes, evidencias sin validar, fin de período)?
5. ¿Qué condiciones debe cumplir una práctica para cerrarse: horas mínimas, informe final, calificación de ambos tutores?
6. ¿El tutor de la entidad accede con una cuenta propia en el sistema o valida por un enlace enviado al correo?

8. Declaración de uso de IA

Los integrantes del equipo usamos asistentes de inteligencia artificial generativa (Claude, Gemini, Pi y OpenCode) para ordenar la estructura, revisar la redacción y generar el código del mapa. Los actores, su clasificación y las preguntas parten del proyecto y de los acuerdos de la ACD1; el equipo revisó y ajustó todo el contenido. Para el concepto de interesado consultamos además a Glinz y Wieringa [6] y a Sommerville [7].

Bibliografía / Referencias

[1] H. Sharp, A. Finkelstein, and G. Galal, "Stakeholder identification in the requirements engineering process," in *Proceedings of the Tenth International Workshop on Database and Expert Systems Applications (DEXA 99)*, IEEE, 1999, pp. 387–391. DOI: 10.1109/DEXA.1999.795198

[2] D. B. Piskala, "Spec-driven development: From code to contract in the age of AI coding assistants," *arXiv preprint arXiv:2602.00180*, 2026. DOI: 10.48550/arXiv.2602.00180

[3] P. Zave and M. Jackson, "Four dark corners of requirements engineering," *ACM Transactions on Software Engineering and Methodology*, vol. 6, no. 1, pp. 1–30, 1997. DOI: 10.1145/237432.237434

[4] K. Wiegers and J. Beatty, *Software Requirements*, 3rd ed. Redmond, WA: Microsoft Press, 2013, ISBN: 978-0-7356-7966-5.

[5] GitHub, *Spec kit: Agentic SDD command reference*, https://github.com/github/spec-kit/blob/main/docs/reference/agentic-sdd.md, Accessed: 2026-10-08, 2026.

[6] M. Glinz and R. J. Wieringa, "Guest editors' introduction: Stakeholders in requirements engineering," *IEEE Software*, vol. 24, no. 2, pp. 18–20, 2007. DOI: 10.1109/MS.2007.42

[7] I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2016, ISBN: 978-0-13-394303-0.